



RISC-V FPGA Implementation



Department of
Electronic and
Computer Engineering

Jingyu Sun

Electronic and Computer Engineering

Introduction

As Moore's Law decelerates, it signifies that increases in total processing power no longer come along with each new fabrication node. The five principles of domain-specific components were proposed and defined by John Hennessy and David Patterson to achieve better area efficiency and energy savings in computing systems.

Under this circumstance, RISC-V emerges as a beacon of innovation, offering a new paradigm in processor architecture. This open standard instruction set architecture (ISA) enables designers to create specialized processors tailored to precise applications, rather than relying on the one-size-fits-all approach of traditional ISAs.

Aim

The project focuses on an in-depth exploration of the RISC-V ISA, with a specific emphasis on developing a single-cycle design for the RV32I Base Integer instruction set using System Verilog.

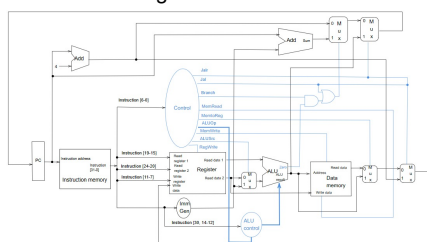
The initial design of the project is tailored to support a specific subset of RV32I instructions, followed by the creation and evaluation of a more sophisticated two-stage pipelined design. The GNU toolchain is used to aid testing. The Vivado tool will be employed for design, validation, and FPGA implementation.

Method

This project is divided into three main phases:

Non-pipelined design:

There are nine core components needed in a non-pipelined design. Instruction Memory unit, Program Counter unit, Add unit, Arithmetic Logical Unit, register unit, Data Memory unit, Immediate Generation unit, Multiplexer unit and Control unit. The schematic depicted below illustrates a simplified Datapath integrated with a control unit, which manages the unconditional branch control signals.

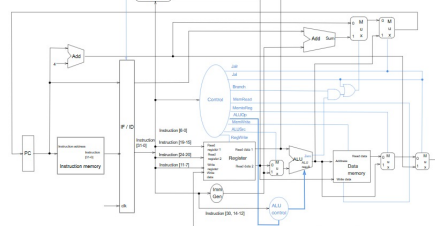


The Datapath with the unconditional branch control signals.

Two-stage pipelined design:

For this two-stage pipelined design, a segment register is inserted between the instruction fetch and the instruction decode stages to concrete pipelined design.

Nevertheless, this design necessitates addressing the control hazard, also known as the branch hazard, to ensure the correct execution sequence of instructions.



A two-stage pipelined Datapath including a hazard detection unit.

FPGA implementation:

A basic numerical counter program, crafted in the C language, has been executed on an FPGA to showcase the functionality of the pipelined design.

Results

This processor architecture designed is expected to execute all R-type instructions, including 10 instructions, part of I-type instructions, comprising 12 instructions, all SB-type instructions, totalling 6 instructions, one and only one UJ-type instruction (jal), one and only one U-type instruction (lui), and all SB-type instructions, consisting of 5 instructions based on RV32I instruction set.

The performance analysis contrasts non-pipelined and two-stage pipelined designs using timing report metrics from Vivado. The single-cycle non-pipelined design shows the worst negative slack (WNS) of 0.151 nanoseconds, suggesting a marginally higher operational frequency of about 101.5 MHz against a nominal 100 MHz (10ns period). Conversely, the two-stage pipelined design demonstrates a WNS of 0.73 nanoseconds, corresponding to an estimated operational frequency of about 107.5 MHz.

Timing	Setup	Hold	Pulse Width
Worst Negative Slack (WNS):	0.151 ns		
Total Negative Slack (TNS):	0 ns		
Number of Failing Endpoints:	0		
Total Number of Endpoints:	25368		

Single-Cycle timing report

Timing	Setup	Hold	Pulse Width
Worst Negative Slack (WNS):	0.73 ns		
Total Negative Slack (TNS):	0 ns		
Number of Failing Endpoints:	0		
Total Number of Endpoints:	2657		

Two-stage pipelined timing report

The C language program displayed is designed to validate the functionality of the two-stage pipeline design in a real-world scenario. The following program is assembly program. It involves the use of a RISC-V cross-compiler to generate the necessary assembly instructions that are then implemented on an FPGA. The accompanying figure illustrates the execution of this C program on the FPGA hardware.

```
int main() {
    int n=1;

    int* led_data = (int*) 0x0; // the
    output of led is the value from the
    first address of data_memory

    while (1){

        *led_data = n; n++;

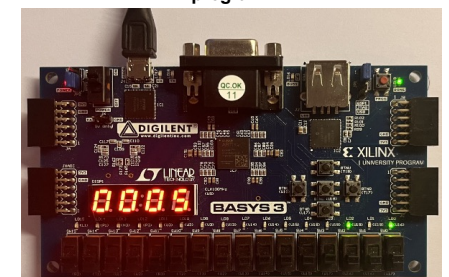
        for (int i=0; i<12500000;
i++);}

    return 0;
}
```

The C program executed on the FPGA to verify its functionality

```
Assembly of section .text
0: 01c01113      addi    sp,sp,32
4: 00010413      li      a5,1          addi a5,a5,1
8: 00170793      sw      a5,-20(a0)
c: 70420023      lw      a5,-20(a0)
10: f0422223      lw      a4,-20(a0)
14: f0427203      sw      a4,-20(a0)
18: 00170793      addi    a5,a5,1
24: 00170793      sw      a5,-20(a0)
28: 70420023      lw      a5,-20(a0)
32: f0422223      sw      a4,-24(a0)
36: 0100000f      jal     ra,10<boot+0x10> jal x1, +10
40: f0427203      sw      a5,-24(a0)
44: 00170793      addi    a5,a5,1
48: 70420023      lw      a5,-24(a0)
52: f0427203      sw      a4,-24(a0)      das
56: 000147b7      lui     a5,0x7b
60: 2170793a      addi    a5,a5,375
64: f0c10003      bge     a5,a4,-24
68: f0c1000f      jal     x2,-68          //jump to 14
```

Assembly code generated by compiling the C program



FPGA demonstration

Conclusion and personal reflection

A fully functional RV32I RISC-V two-stage pipelined processor has been designed and demonstrated in a real-world C language example.

Through my studies, I have gained substantial knowledge in processor design, particularly in utilizing Vivado's suite of tools for debugging HDL designs. Furthermore, I've deepened my understanding of the relationship between C programs and hardware, as well as the translation of compiled programs into assembly instructions.

Acknowledgements

I would like to thank my supervisor Dr Richard Conway for all of his help and guidance with this project.

